Student Performance Prediction Using ML

PRESENTATION 2026

Student Name

Student Name

M.Jyothi

T.Nandini

SK.Sadiya

M.Nandhini

Student Email

jyothimanda49@gmail.com

thumunandini49@gmail.com

shaiksadiya1811@gmail.com

mood.nandhini2005@gmail.com

Guide

Guide Name

Rapaka Manoj Kumar

OUTLINE

- Abstract of the Project
- Problem Statement
- Proposed Solution
- System Architecture
- Live Demo of the Project
- Embedded Video of Project
- Conclusion
- Future Scope

Abstract

- The Student Performance Prediction project uses machine learning techniques to analyze academic and personal factors that influence student performance and predict their expected academic outcomes.
- It uses a machine learning pipeline that collects and preprocesses student data, identifies important performance-related features, and trains suitable ML models for prediction.
- The system can predict student performance based on factors such as attendance, previous marks, study time, assignments, and other academic attributes, helping to identify students who may need additional support.
- The proposed system can assist teachers and educational institutions in monitoring student progress, identifying learning gaps, and taking early steps to improve academic performance while supporting data-driven decision-making.

Problem Statement

- Student performance prediction is a critical challenge in education. Many factors such as attendance, study habits, previous marks, and participation influence academic outcomes. Identifying at-risk students early is difficult without a data-driven approach.
- Manual evaluation methods are time-consuming, subjective, and may not capture hidden patterns in data. Institutions need intelligent systems that can analyze student data and predict performance accurately to support better academic planning.
- Existing traditional methods cannot effectively handle large datasets or adapt to diverse student profiles.

Proposed Solution

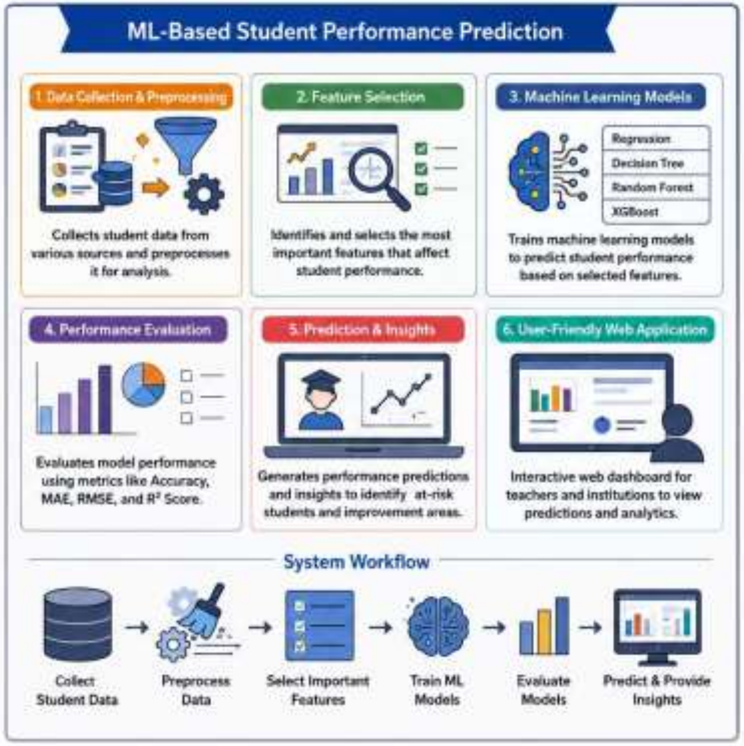
Enhances student performance prediction using Machine Learning and data analytics techniques while keeping it accurate, reliable, and user-friendly.

Data Collection & Preprocessing: Collects student data from multiple sources and preprocesses it by handling missing values, encoding categorical features, and normalizing data for accurate analysis.

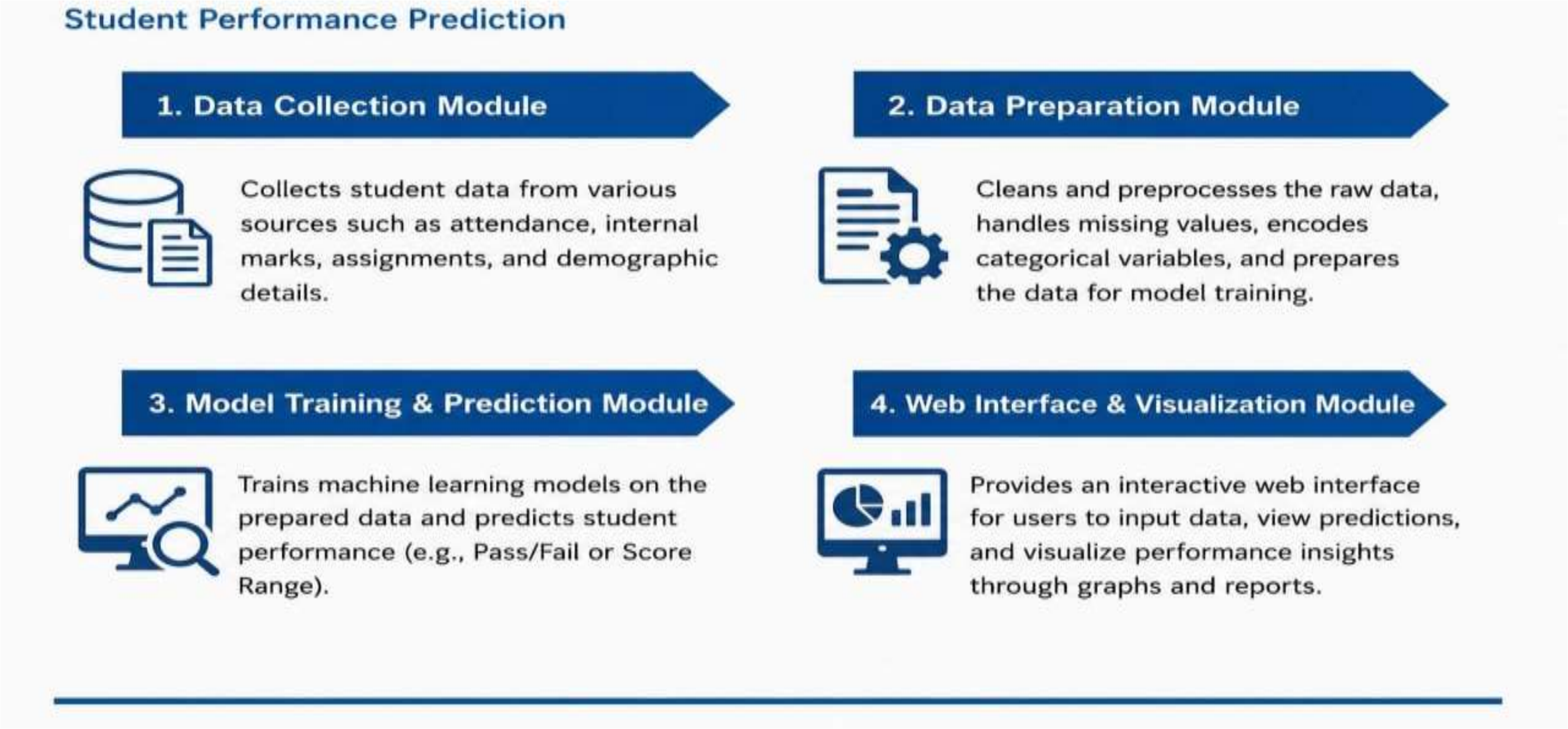
Feature Selection: Identifies and selects the most important features such as attendance, study time, previous marks, assignments, and participation that influence student performance.

Machine Learning Models: Applies ML algorithms like Regression, Decision Trees, Random Forest, and XGBoost to train models and predict student performance scores or grades.

Performance Evaluation: Evaluates models using metrics such as Accuracy, MAE, RMSE, and R² Score to ensure reliable and accurate predictions.



System Architecture



RANDOM FOREST

Random Forest is an **Ensemble** Machine Learning algorithm that builds multiple decision trees during training and outputs the mode of the classes (classification) or mean prediction (regression) of the individual trees. It is widely used for prediction tasks due to its high accuracy, robustness, and ability to handle large datasets with higher dimensionality.

Key Components

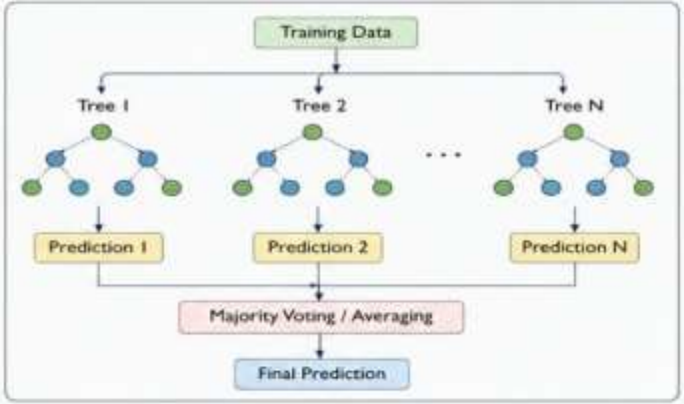
Multiple Decision Trees: Random Forest constructs multiple decision trees using different subsets of the training data.

Random Feature Selection: At each split in a tree, a random subset of features is considered, which reduces correlation and improves generalization.

Ensemble Learning: The final prediction is made by aggregating the predictions of all individual trees (majority voting for classification or averaging for regression).

Robust and Accurate: Random Forest reduces overfitting and works well even with noisy data and outliers.

How Random Forest Works?



Workflow in Student Performance Prediction

WhatsApp | Home | student_analysis

localhost:8888/notebooks/Desktop/student_analysis.ipynb

jupyter student_analysis Last Checkpoint: 16 hours ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

[1]:
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings("ignore")

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

Load Dataset

[2]:
```python
df = pd.read_csv("student_performance_1000_cleaned.csv")

print("Dataset loaded successfully!")
```

```
Dataset loaded successfully!
```

[3]:
```python
df.head()
```

[3]:

	Student_ID	Age	Gender	Study_Hours	Attendance	Previous_Score	Assignments_Score	Midterm_Score	Sleep_Hours	Extracurricular	Parental_Education	Study_Me
0	1	20	Female	2	80	43	67	91	5.6	Yes	School	C
1	2	21	Male	4	70	58	63	92	5.8	No	Intermediate	C
2	3	19	Female	2	59	42	47	39	6.9	Yes	School	Self

Search ENG IN 11:31 14-08-2026

Run Kernel Settings Help

Code JupyterLab Python

```
nt(df.columns)
```

```
ex(['Student_ID', 'Age', 'Gender', 'Study_Hours', 'Attendance',
    'Previous_Score', 'Assignments_Score', 'Midterm_Score', 'Sleep_Hours',
    'Extracurricular', 'Parental_Education', 'Study_Method',
    'Internet_Access', 'Parental_Support', 'Stress_Level',
    'Motivation_Level', 'Participation', 'Final_Score', 'Performance'],
   dtype='str')
```

```
nfo()
```

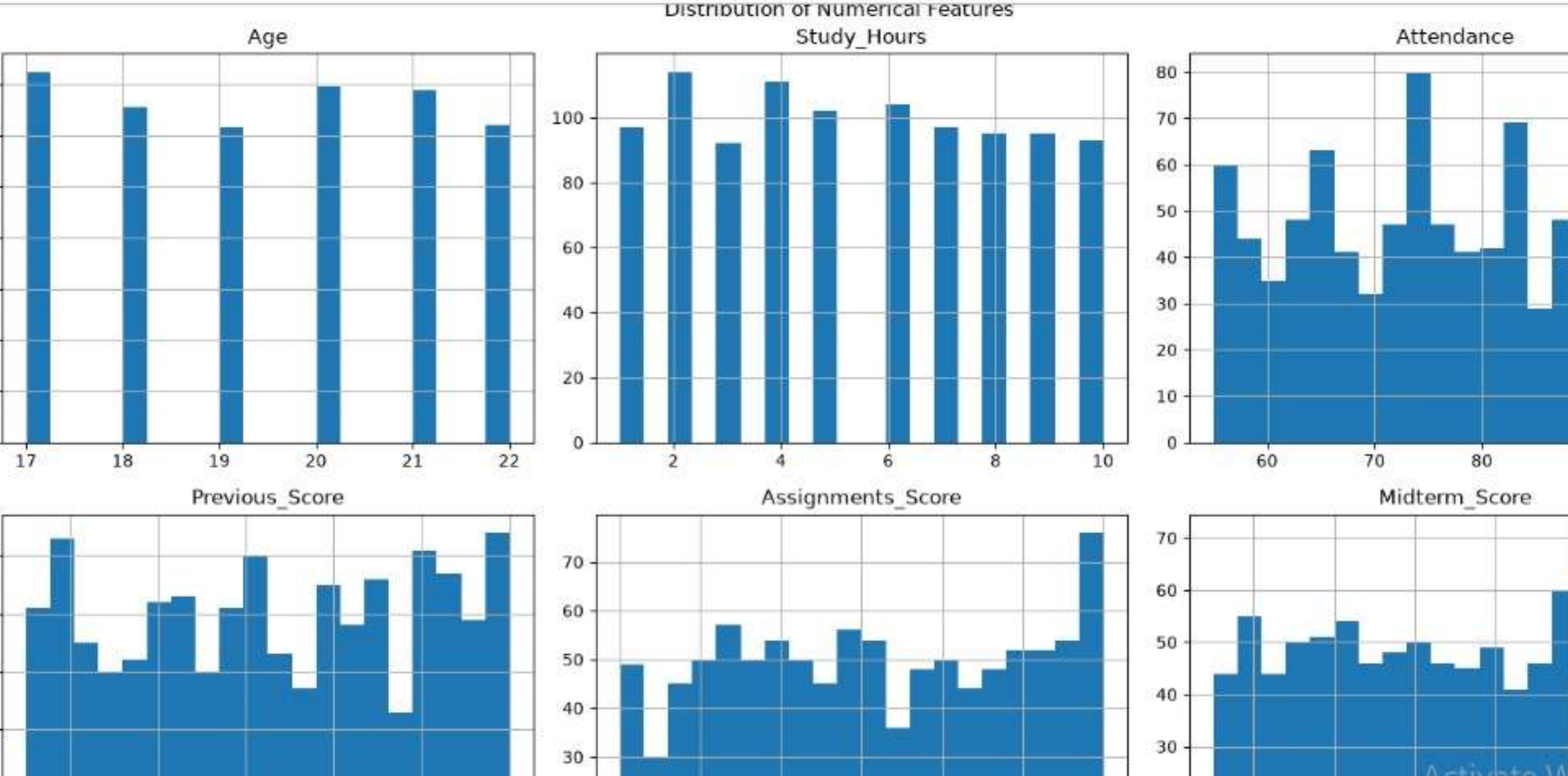
```
ass 'pandas.DataFrame'>
geIndex: 1000 entries, 0 to 999
a columns (total 19 columns):
 Column              Non-Null Count  Dtype
 ------              --------------  -----
 Student_ID          1000 non-null   int64
 Age                 1000 non-null   int64
 Gender              1000 non-null   str
 Study_Hours         1000 non-null   int64
 Attendance          1000 non-null   int64
 Previous_Score      1000 non-null   int64
 Assignments_Score   1000 non-null   int64
 Midterm_Score       1000 non-null   int64
 Sleep_Hours         1000 non-null   float64
 Extracurricular     1000 non-null   str
 Parental_Education  1000 non-null   str
 Study_Method        1000 non-null   str
```

Run Kernel Settings Help

Code

JupyterLab

Python



Run Kernel Settings Help

Code

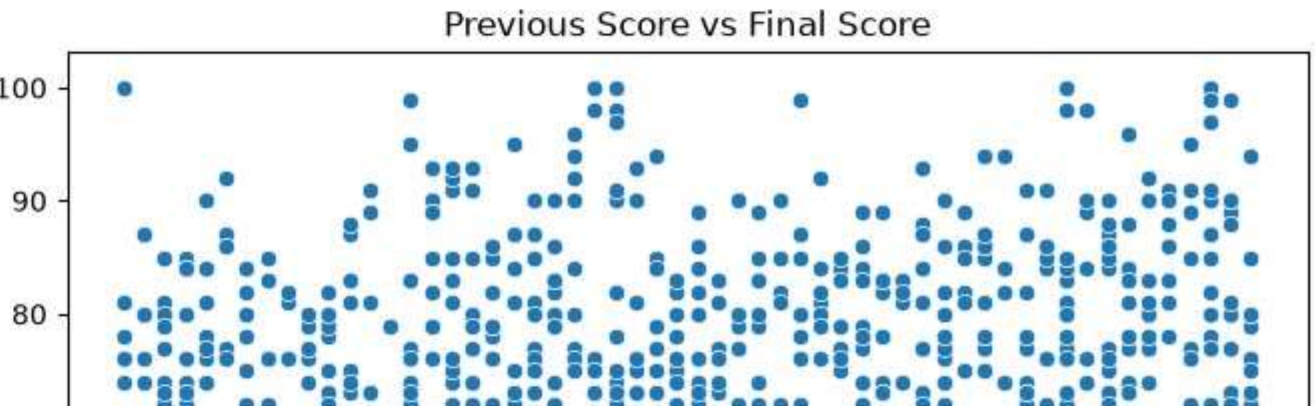
JupyterLab Python

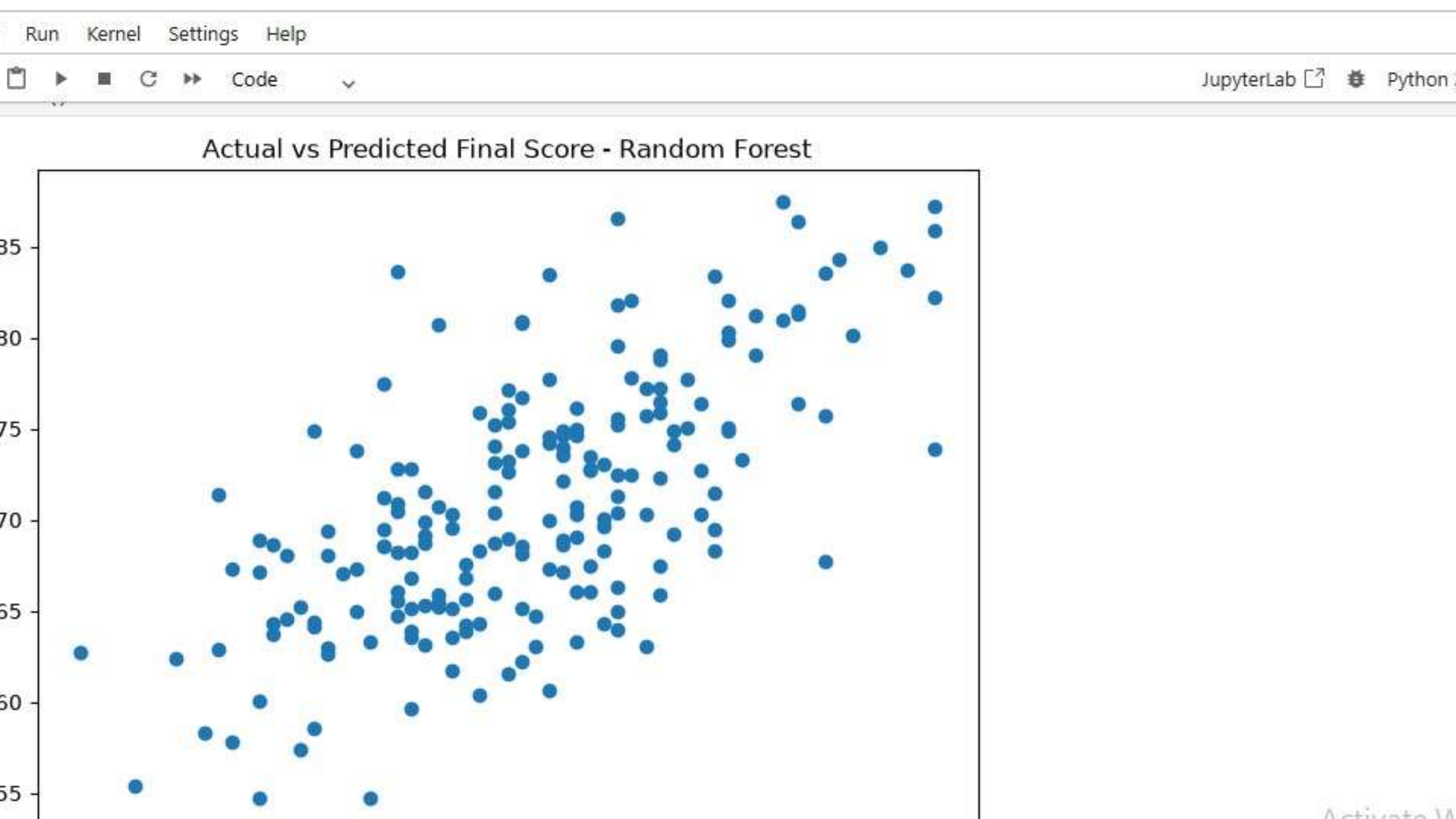
```
evious vs Final Score
figure(figsize=(8, 5))

scatterplot(
data=df,
x="Previous_Score",
y="Final_Score"


title("Previous Score vs Final Score")
xlabel("Previous Score")
ylabel("Final Score")

show()
```




Run
Kernel
Settings
Help
Code
JupyterLab
Python
Actual vs Predicted Final Score - Random Forest

Performance Evaluation Metrics

(Student Performance Prediction)

1. Accuracy

Used to measure how many student performance predictions are correctly predicted by the model out of total predictions.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Where,

- TP = True Positives (Correctly predicted Pass)
- TN = True Negatives (Correctly predicted Fail)
- FP = False Positives (Incorrectly predicted Pass)
- FN = False Negatives (Incorrectly predicted Fail)

2. Categorical Cross-Entropy Loss

Used during training to measure the difference between predicted probabilities and actual labels.

$$\text{Loss} = -\sum_{i=1}^{N} y_i \log(p_i)$$

Where,

- y_i = Actual label (0 or 1)
- p_i = Predicted probability
- N = Number of classes

Why These Metrics? Accuracy helps us understand overall correctness of predictions, while Cross-Entropy Loss helps the model learn better by minimizing prediction errors during training.

Performance Evaluation Metrics

(Student Performance Prediction)

3. Training Accuracy

Shows how well the CNN learns from the generated CAPTCHA dataset.

$$\text{Training Accuracy} = \frac{\text{Correct Predictions on Training Data}}{\text{Total Training Samples}}$$

4. Validation Accuracy

Used to check how well the CNN generalizes to unseen CAPTCHA images.

$$\text{Validation Accuracy} = \frac{\text{Correct Predictions on Validation Data}}{\text{Total Validation Samples}}$$

5. Training Loss

Helps detect overfitting if validation loss increases while training loss decreases.

$$\text{Training Loss} = -\sum_{i} y_i \log(\hat{y}_i)$$

These metrics help evaluate the performance of the CNN model during training and validation, ensuring the model learns effectively and generalizes well to new data.

Performance Evaluation Metrics

(Student Performance Prediction)

6. Validation Loss

Helps detect overfitting if validation loss increases while training loss decreases.

$$Validation\ Loss = -\sum_{i} y_i \log(\hat{y}_i)$$

7. Adversarial Robustness Evaluation

Measures how well the CNN performs when CAPTCHA images are modified with adversarial noise.

$$Adversarial\ Accuracy = \frac{Correct\ Predictions\ on\ Adversarial\ Images}{Total\ Adversarial\ Images}$$

FGSM Noise Formula Used:

$$x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

Where,

- x = Original image
- ϵ = Small perturbation value
- $\nabla_x J$ = Gradient of loss w.r.t input
- x_{adv} = Adversarial image

KEY INSIGHTS

- The **Student Performance Prediction** using Machine Learning project provides the following insights:
- **Study hours** have an important relationship with students' final scores.
- **Attendance** plays a significant role in academic performance.
- **Previous, assignment, and midterm scores** are useful indicators for predicting final performance.
- Students with higher **motivation** and **parental support** generally show better performance.
- High **stress levels** can negatively affect academic outcomes.
- Different **study methods** may lead to differences in student performance.
- **Machine Learning** can identify patterns among multiple student-related factors and use them to predict the final scores accurately.

RECOMMENDATIONS

- Students should maintain regular **study schedules** and focus on **consistent preparation**.
- Colleges should encourage regular **attendance** and **active classroom participation**.
- Students with low previous or midterm scores should receive **early academic support**.
- Teachers can provide **personalized guidance** based on students' performance patterns.
- Students should be encouraged to maintain a **healthy balance** between study, sleep, and stress management.
- Parents and teachers can provide **additional support** to students who show declining performance.
- The dataset can be **expanded** with more real-world student records to improve model reliability.
- Institutions should use predictive insights to design effective **intervention strategies** for student success.

Conclusion

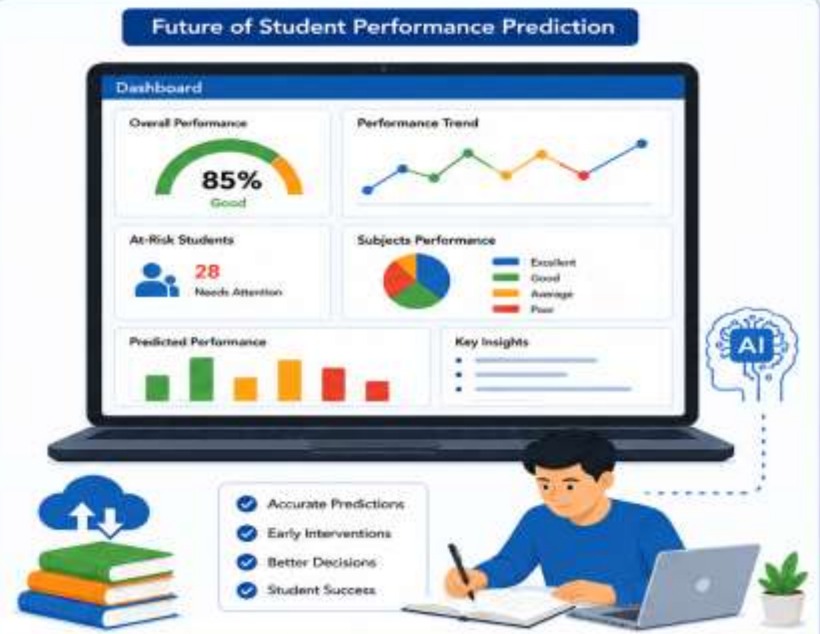
- The project successfully enhances traditional student evaluation methods using a **Machine Learning-based approach** for Student Performance Prediction.
- Integration of data analytics and ML models improves the **accuracy and reliability** of performance predictions.
- The system helps educators identify **at-risk students** early and take proactive measures to improve academic outcomes.
- The user-friendly web application enables **real-time predictions, analysis, and visualization** for better decision-making.
- The project achieves a strong balance between **usability, prediction accuracy, and model robustness.**
- Overall, the ML-based approach outperforms traditional methods and provides a **scalable, efficient, and intelligent solution** for improving student success.

ML-Based Student Performance Prediction

- ✔ Accurate performance prediction using ML models
- ✔ Early identification of at-risk students
- ✔ Data-driven insights for better academic decisions
- ✔ Supports student success and institutional growth

Future Scope
(Student Performance Prediction)

- **Expand to diverse datasets:** Include data from different institutions, regions, and countries to improve model generalization and reliability.
- **Speech and voice analytics:** Use voice-based inputs and feedback to support students with disabilities and enhance engagement.
- **Real-time monitoring and alerts:** Build real-time systems to continuously track performance and send alerts for at-risk students.
- **LMS integration:** Seamlessly integrate with Learning Management Systems to automate data flow and personalize learning paths.
- **Cloud-based solution:** Deploy on cloud platforms for better scalability, high availability, and secure access from anywhere.
- **Advanced AI models:** Use advanced techniques like deep learning, ensemble learning, and reinforcement learning to improve prediction accuracy.
- **Explainable AI (XAI):** Incorporate XAI techniques to understand model decisions and increase transparency and trust.
- **Advanced dashboards and reports:** Develop interactive dashboards and reports to provide actionable insights for teachers, parents, and administrators.



These advancements will help build a smarter, more inclusive, and more effective student performance prediction system.

Thank You